

Домашнее задание 10

Лиза Фоменко

Турнир - это полный ориентированный граф, то есть такой ориентированный граф, в котором между любой парой различных вершин есть ровно одно ребро.

Задание 1

Докажите, что в турнире на n вершинах есть простой (несамопересекающийся) путь длины $n - 1$. Постройте алгоритм, находящий такой путь, и оцените время его работы.

Попробуем доказать индуктивно.

База индукции: для $n = 2$ очевидно: у нас есть ребро $v_1 \rightarrow v_2$, то есть путь длины $n - 1 = 1$.

Предположение индукции: Пусть для турнира с $n - 1$ вершиной верно, что существует простой путь длины $n - 2$

Шаг индукции: Рассмотрим турнир из n вершин. Заметим, что если убрать из него случайную вершину v вместе с выходящими/входящими в нее ребрами, то мы получим турнир из $n - 1$ вершин, для которого известно, что существует такой путь из $n - 2$ вершин вида $v_1, \dots, v_{n-1}$.

Попробуем вернуть вершину v в граф. Пойдем от вершины v_1 по имеющемуся простому циклу длины $n - 2$, то есть $v_1, v_2, v_3, \dots$. Для каждой вершины проверяем, существует ли ребро v, v_i ("вход" v в цикл): заметим, что если его не существует, то обязательно существует v_i, v ("выход" из цикла в v). Найдем первую вершину, для которой существует v, v_i - через него v войдет в цикл". Так как мы проверяем каждую вершину по циклу, то для всех вершин, находящихся раньше найденной v_i , существует ребро, ведущее в v . Тогда мы можем "убрать" из нашего пути ребро v_{i-1}, v_i и добавить в него два ребра: v_{i-1}, v и v, v_i . Тогда вершина v войдет в путь, а его длина увеличится на 1: получим путь из n вершин длины $n - 1$ - простой путь.

Перейдем к алгоритму поиска. На примере доказательства по индукции мы можем точно так же построить алгоритм поиска, добавляя по одной вершине за ход алгоритма.

$G = (V, E)$ # наш граф. здесь не оговорено как он задан

$v_0 = G[0]$ # пример выбора начальной вершины

$v_1 = v_0_merged[0]$ # выбираем вторую вершину: одна из тех, в которые существует ребро из v_0 : $v_0 \rightarrow v_1$

$path = [v_0, v_1]$

```
for v in V - v0 - v1:
    if v not in path:
        for i in range(len(path)):
            if v -> path[i]:
                path = path[:i] + v + path[i:]
                break
```

return path

Этот алгоритм работает за $O(n^2)$: мы проходим цикл для каждой вершины ($n - 2, O(n)$), и про-
сматриваем вершины из уже существующего пути, что тоже занимает $O(n) \Rightarrow$ асимптотика $O(n^2)$. Его
корректность доказана при доказательстве существования самого пути :)

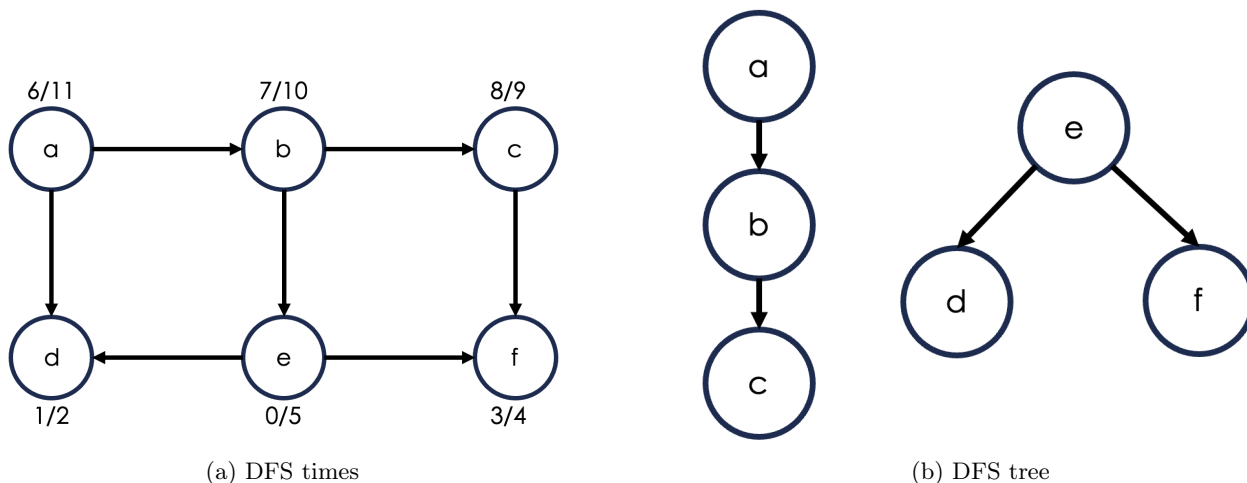
Задание 2

Примените DFS к графу, начиная с вершины e . Найдите время открытия и закрытия каждой вершины, приведите лес обхода в глубину. Для рисования графов рекомендуется использовать [вот эту штуку](#).

Выпишем порядок вершин: a, b, c, d, e, f. Обход начинается в e, идем в d, из нее никуда пойти не можем => возвращаемся в e и проходим в f, из которой путей тоже не существует. Получили первое дерево обхода.

Для второго дерева обхода начнем с вершины a (так как идем в алфавитном порядке), затем идем через b в c. Второе дерево обхода готово.

У меня люто висел сайт который был рекомендован, поэтому я вместо борьбы с латехом нарисовала картинки за 5 минут и прикрепляю их в виде .png сюда)



Задание 3

Постройте алгоритм, находящий все стоки в ориентированном графе, если известно, что их $\Theta(\sqrt{|V|})$. Сток - это вершина с нулевой исходящей степенью. Граф уже лежит в памяти в виде матрицы смежности.

Пусть в матрице исходящие вершины находятся в строке (чтобы оперировать можно было количеством столбцов). Наивный алгоритм работает за $O(n^2)$ - при просмотре каждого элемента матрицы смежности. Но тут никак не учитывается количество стоков. Поэтому попробуем ускорить алгоритм, используя это великое знание.

Идея: делим матрицу на две части по количеству столбцов: α и $n - \alpha$. В первой итерации алгоритма мы будем рассматривать только первые αn столбцов. Также нам нет никакой необходимости идти по всем столбцам для вершины, если мы уже встретили 1 в матрице в этом ряду. Собственно что мы будем делать: для всех $|V| = n$ вершин смотрим первые αn столбцов. Идем по столбцам для каждой вершины, если встречаем 1, выкидываем вершину из кандидатов в стоки. После полного прохода, который займет $O(n * \alpha)$, у нас останется набор вершин-кандидатов в стоки размера n' , для которых на первых α столбцах находятся только 0. Далее только для этих вершин мы будем аналогичным образом рассматривать $n - \alpha$ столбцов. Этот этап у нас займет $O(n' * (n - \alpha))$. Итоговая асимптотика: $O(n * \alpha) + n' * (n - \alpha)$. Мы знаем, что она должна быть меньше $O(n^2)$, при этом n в явном виде входит в оба слагаемых. Тут я приведу рассуждения, которые не могу обосновать математически, но они мне кажутся по наитию верными. У нас после первого шага остается n' кандидатов. Мы знаем, что всего стоков $\Theta(\sqrt{|V|}) = \Theta(\sqrt{n})$. Тогда мы можем грубо оценить n' через $O(\sqrt{n})$, и тогда асимптотика второй части алгоритма будет $O(\sqrt{n} * (n - \alpha))$, что приводит нас к $O(n\sqrt{n})$. Можно ли подобрать α так, чтобы первая часть алгоритма тоже была $O(n\sqrt{n})$? Можно, при $\alpha = \sqrt{n}$. Тогда итоговая сложность составляет $O(n\sqrt{n})$.

Можно ли быстрее? Могут ли обе части работать быстрее? Мне кажется нет, потому что во второй части алгоритма у нас всегда будет зависимость от $\Theta(\sqrt{n})$. При делении столбцов у нас всегда останется $1 - \alpha$ столбцов, что всегда при $\alpha \ll n$ будет линейной. При $\alpha = n$ мы получим $O(n^2)$ в первой части алгоритма.